

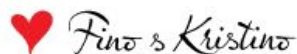
Christian Nagel

Microsoft MVP

C# 14 and .NET 10 – What's new?



We would like to thank our sponsors



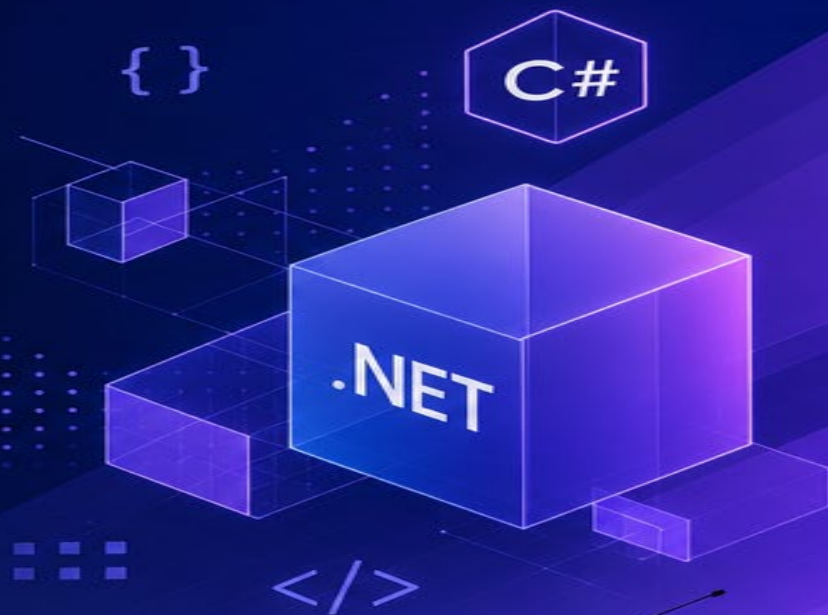
THRIVE 2026
RIMSKE TERME, SLOVENIA

C# 14 and .NET 10 – What's New?

The next evolution for developers



Building the future.
Together.



THRIVE
2026

Christian Nagel

- 20+ years of experience with C# and .NET
- Author of 20+ books on C# and .NET (latest: "Modern C# in .NET 10")
- Microsoft MVP for C# and .NET
- Speaker at global conferences (TechEd, BASTA!, Thrive... for 20+ years)
- Passionate about language design and developer productivity



Topics

Highlights of .NET and C# updates

1. Libraries - Cryptography
2. Blazor - the direction where ASP.NET Core UI moves to
3. C# Updates - Spans and extensions
4. File-based Apps
5. Native AOT
6. Runtime Async

1. Post Quantum Cryptography

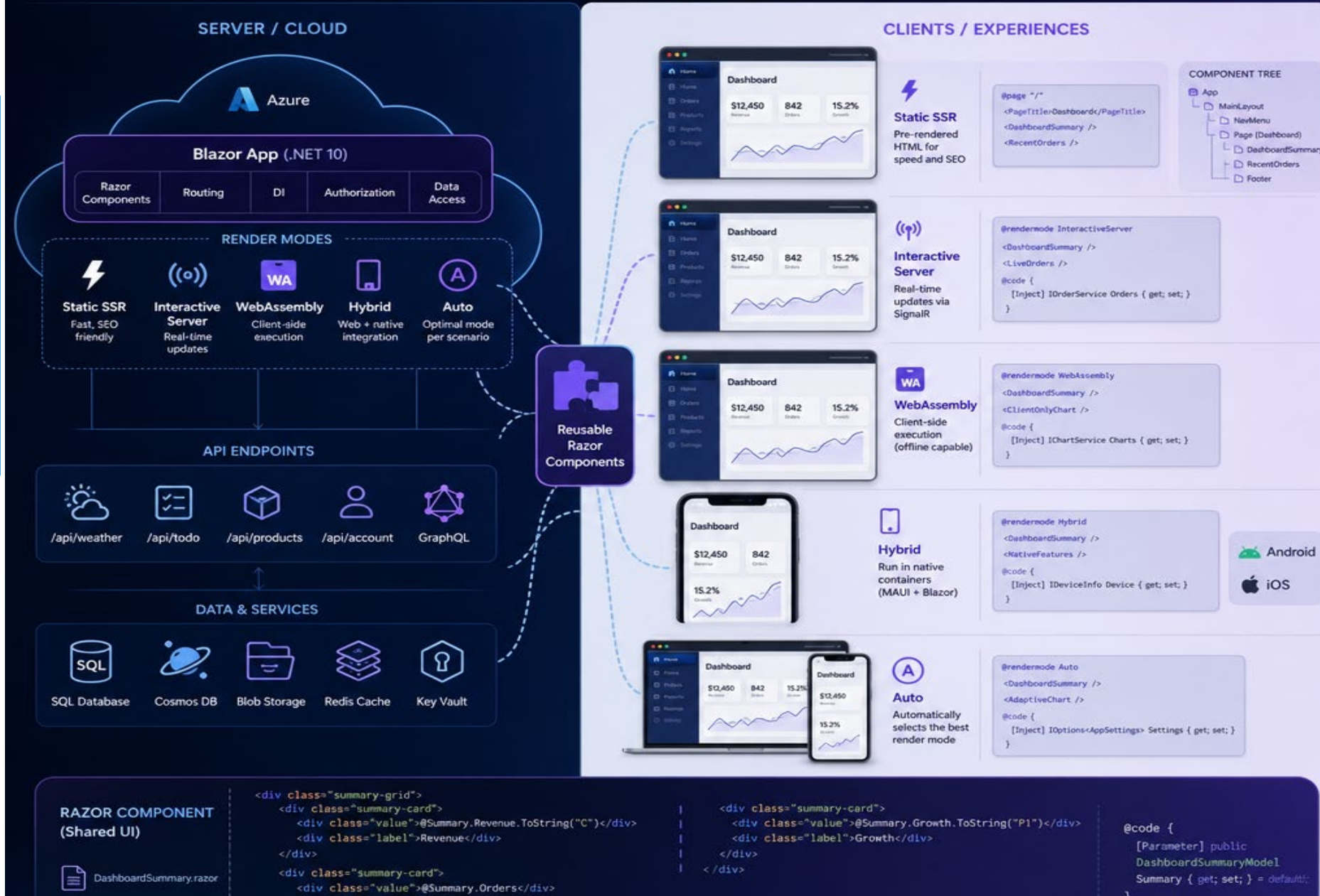
Quantum computing breaks current cryptography!

.NET 10 offers post-quantum algorithms for secure communication.



2. Blazor

ASP.NET Core component- based UI Architecture



Blazor Features

1. Blazor SSR
2. Blazor Server
3. Blazor WebAssembly (WASM)
4. Blazor Hybrid
5. Blazor Auto - switch SSR, Server, WASM

Blazor - What's new with ASP.NET Core 10

- Declarative persisting component state
- QuickGrid enhancements
- HttpClient response streaming enabled by default

Building the Bridge from C# 13 to C# 14 and 15

Continuing the journey of modern, expressive, and high-performance C#

C# 13

-  Params Collections
-  Improved Lambda Expressions
-  Ref Fields (in ref structs)
-  Overload Resolution Improvements
-  Experimental Features and More

C# 14

- ### C# 14
-  Extension Everything
 -  Partial Properties and Indexers
 -  Interceptors
 -  Enhanced Pattern Matching
 -  Performance and Tooling Boosts
 -  And More to Come

C# 15

-  Further Language Evolution
-  Developer Productivity Elevated
-  AI-Friendly by Design
-  Cloud-Native and Distributed Advancements
-  Performance Without Compromise
-  And Beyond...

OUR FOUNDATION



Simplicity
Making powerful things easy to use.



Expressiveness
Write what you mean, clearly.



Performance
Fast today, faster tomorrow.



Safety
Strong by default, secure by design.



Community
Built together, for everyone.

One language. Two bridges. Endless possibilities.

The future of C# is built together. ❤️

C# 13 — params Collections

Extended beyond arrays — now accepts any collection type

```
// C# 12 and earlier — only arrays
void Log(params string[] messages) { }

// C# 13 — any collection type
void Log(params IEnumerable<string> messages) { }
void Log(params ReadOnlySpan<string> messages) { } // zero-alloc!
void Log(params List<string> messages) { }

// Call site looks identical
Log("Starting", "Processing", "Done");
```

Why it matters for C# 14+: Paves the way for **implicit Span conversions** and zero-allocation APIs.

C# 13 Partial Properties - C# 14 field keyword

partial keyword extended to properties

```
// Declaration (e.g., in designer file)
public partial class UserViewModel
{
    public partial string Name { get; set; }
    public partial int Age { get; set; }
}
```

Foundation for: field keyword in C# 14 and source generator patterns.

```
// Implementation (e.g., in your file)
public partial class UserViewModel
{
    public partial string Name
    {
        get;
        set => field = value.Trim();
    }
    public partial int Age
    {
        get;
        set => field = Math.Max(0, value);
    }
}
```


field keyword — Why It Matters

Direct access to the compiler-generated backing field

***Tip:** Source generators (e.g., MVVM Toolkit) use `partial` + `field` to eliminate all boilerplate.*

C# 13 — Lock Type

New System.Threading.Lock — replaces lock(object)

```
// Before C# 13
private readonly object _lock = new();
lock (_lock) { /* critical section */ }

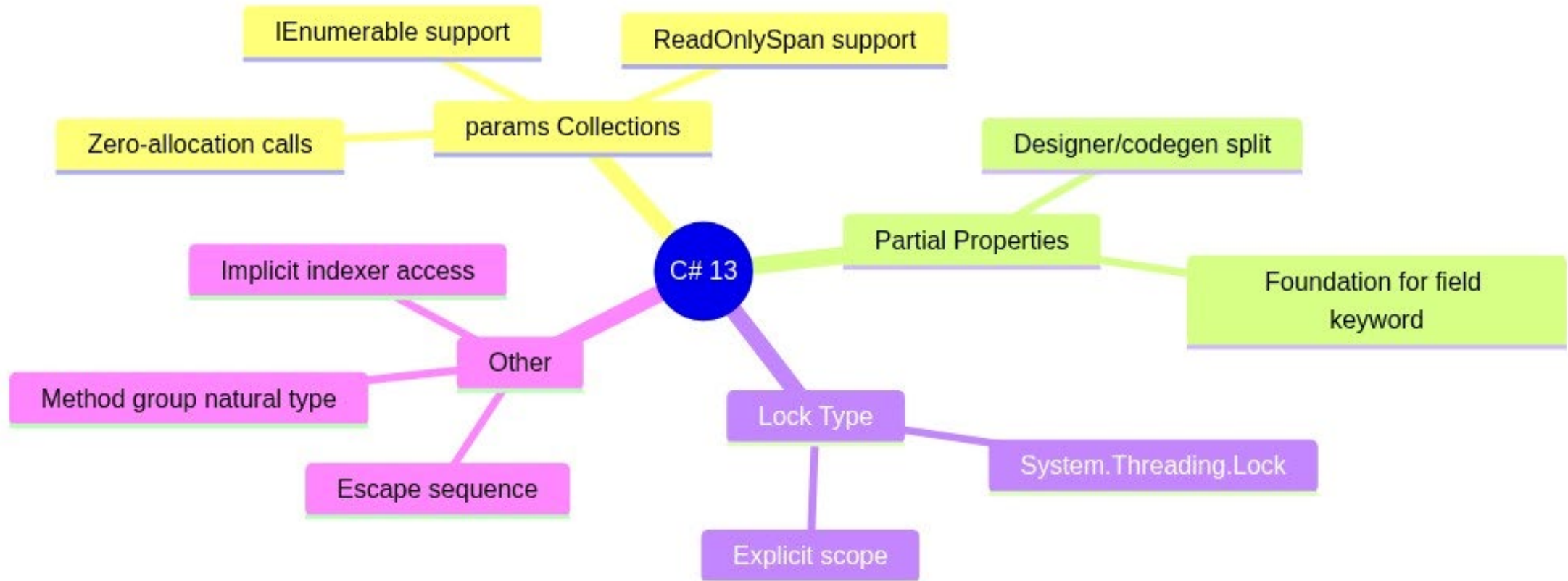
// C# 13 – dedicated Lock type
private readonly Lock _lock = new();
lock (_lock) { /* critical section */ }

// Lock also supports explicit scope
using (_lock.EnterScope())
{
    // critical section
} // automatically released
```

Benefit: Better diagnostics, reduced memory thread-safety improvements.

Why it matters for C# 14+: Along the way of async performance improvements!

C# 13 Feature Summary



C# 14

*Released with .NET 10 —
Productivity & Performance*

PRODUCTIVITY & PERFORMANCE

Productivity is doing more of what matters.
Performance is doing it better.



FOCUS

Prioritize what truly matters.



PLAN

Organize your day with purpose.



EXECUTE

Take action and stay consistent.



MEASURE

Track progress and analyze results.



IMPROVE

Learn, adapt, and keep growing.



C# 14 Feature Overview

C# 14 Features — Impact vs. Adoption Effort



Span — Stack-Only Memory Views

Zero-allocation abstraction over contiguous memory — arrays, stack buffers, unmanaged memory

Reduce GC pressure and improve performance in high-throughput scenarios.

Why Span Matters

Traditional arrays force allocation and copying:

```
// ❌ Problem: Every operation allocates a new array
public void ProcessData(int[] data)
{
    int[] slice = data[10..20]; // ❌ NEW array allocated, data copied
    Console.WriteLine(slice.Length);
}

// Caller
int[] numbers = new int[1000];
ProcessData(numbers); // Allocation + copy overhead
```

Span solves this:

```
// ✅ Solution: Zero-copy view of memory
public void ProcessData(Span<int> data)
{
    Span<int> slice = data[10..20]; // ✅ No allocation, just a view
    Console.WriteLine(slice.Length);
}

// Caller – works with arrays, stacks, unmanaged memory
int[] numbers = new int[1000];
ProcessData(numbers); // ✅ No copy needed
```

Span Core Functionality

Feature	Purpose
Stack-only	Can't be stored in fields; forces efficient short-lived usage
Zero heap allocation	Just a pointer + length — 16 bytes on 64-bit
Indexing & slicing	span[i], span[2..5] like arrays
Iteration	foreach (var item in span)
Write access	Span<T> allows modifications; ReadOnlySpan<T> doesn't

Span Real-World Example: Performance Impact

```
// Without Span – allocates new arrays
public int SumArray(int[] data)
{
    int[] filtered = data.Where(x => x > 10).ToArray(); // Allocation!
    int[] squared = filtered.Select(x => x * x).ToArray(); // Allocation!
    return filtered.Sum();
}

// With Span – zero allocations
public int SumSpan(Span<int> data)
{
    int sum = 0;
    foreach (var x in data)
    {
        if (x > 10)
            sum += x * x;
    }
    return sum;
}

// Benchmark: 100,000 items
// SumArray: 15ms, 400KB allocated
// SumSpan: 2ms, 0KB allocated ← 7x faster, zero GC pressure!
```


C# 14 Enhancement: Implicit Span Conversions

Arrays automatically convert to Span<T> — no .AsSpan() needed

```
void ProcessData(ReadOnlySpan<int> data)
{
    foreach (var item in data)
        Console.Write(item);
}

int[] numbers = [1, 2, 3, 4, 5];
ProcessData(numbers); // C# 14: implicit conversion ✓ (C# 13: needed .AsSpan())

// Works with stack buffers too
Span<byte> buffer = stackalloc byte[256];
ProcessBuffer(buffer); // No copying, zero allocation
```

Why Not Use Span Everywhere?

⚠ Stack-only constraint is intentional:

- Can't be stored in fields → forces disciplined use
- Can't be used in collections → encourages efficient scope boundaries
- **Trade-off:** Better performance, but stricter API design

Best practices:

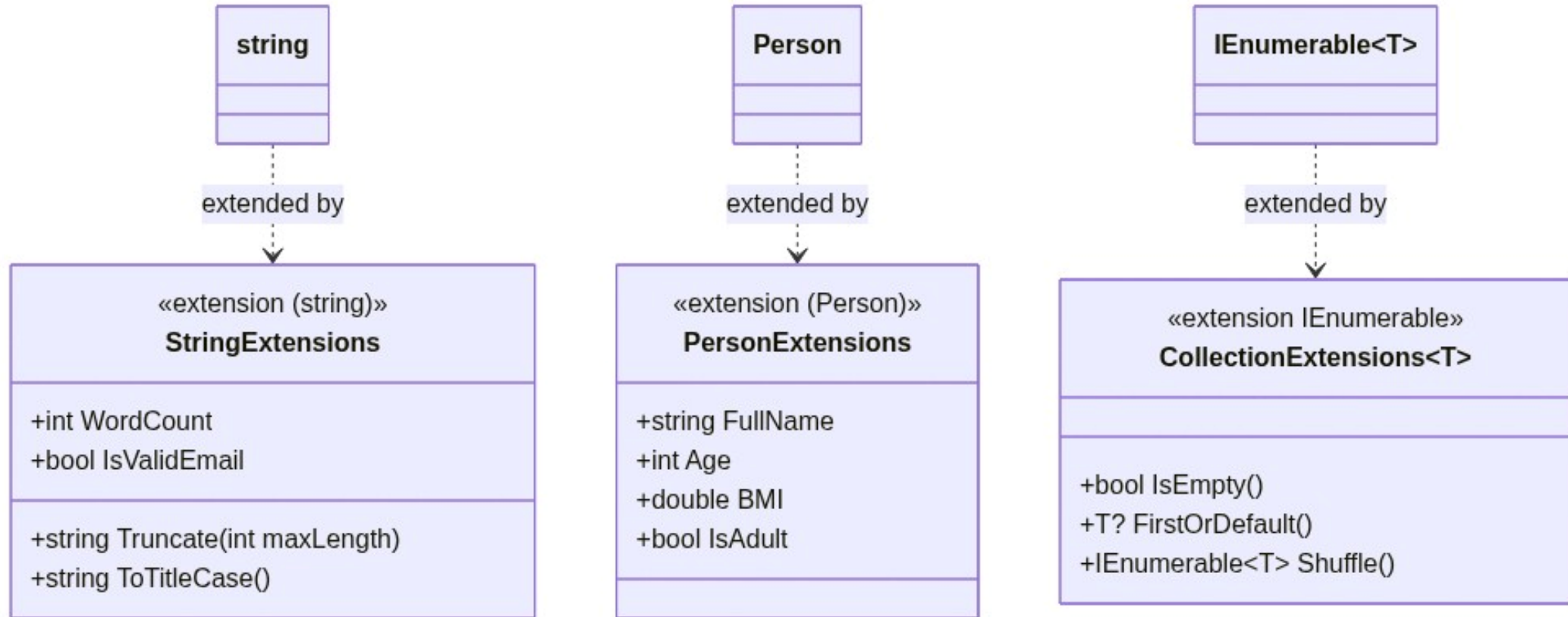
-  Method parameters for high-performance paths
-  Local operations (parsing, validation, transformation)
-  Interop with unmanaged memory
-  Not for long-lived state in classes
-  Not as collection elements

Extensions

For many years we implemented and used extension methods

Now enhanced with properties, operators, static methods

Extension Members — Architecture



4. File-Based Apps (dotnet run app.cs)

Run C# without a project file — .NET 10

```
dotnet run hello.cs
```

```
// hello.cs – no .csproj needed!  
Console.WriteLine("Hello, .NET 10!");  
  
var now = DateTimeOffset.UtcNow;  
Console.WriteLine($"Running at {now:HH:mm:ss}");
```

-  Top-level statements — no Main, no namespace boilerplate
-  Full .NET runtime and NuGet ecosystem available
-  IntelliSense in VS Code with C# Dev Kit
-  Migrate when ready: dotnet project convert hello.cs

File-Based Apps — Directives (1)

Add NuGet packages and alternate SDKs inline:

```
// api.cs
#::sdk Microsoft.NET.Sdk.Web
#::package Microsoft.AspNetCore.OpenApi@10.0.0

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddOpenApi();
var app = builder.Build();

app.MapGet("/", () => "Hello from a single file!");
app.MapGet("/time", () => DateTimeOffset.UtcNow);

app.Run();
```

```
dotnet run api.cs
```


File-Based Apps — Directives (2)

Directive	Purpose
<code>#:package Name@Version</code>	Add a NuGet package reference
<code>#:sdk SdkName</code>	Use alternate SDK (e.g., Web, Worker)
<code>#:property Name=Value</code>	Set an MSBuild property

File-Based Apps - She-Bang with Linux

```
#!/usr/bin/env dotnet  
Console.WriteLine("Hello from a she-bang script!");
```

```
chmod +x hello.cs
```

5. Native AOT (1)

Ahead-of-Time Compilation — Compile .NET apps to native code before deployment

The Problem with Traditional .NET

```
// Traditional JIT compilation model:  
dotnet MyApp.dll  
  
// Runtime startup:  
1. Load .NET runtime           ← ~50-100ms  
2. Load assemblies             ← ~100-200ms  
3. JIT compile hot methods     ← ~200-500ms  
4. Execute                     ← Finally!  
  
// Total: 350-800ms startup time, ~100-200MB memory footprint
```

5. Native AOT (2)

Native AOT Advantage

```
// With Native AOT:  
MyApp.exe (compiled once during build)  
  
// Runtime startup:  
1. Execute native code directly ← ~5-10ms  
2. Done!  
  
// Result: 10-50ms startup time, ~20-50MB memory footprint  
// ✅ 10-50x faster startup, 75-80% less memory!
```

Why Native AOT Matters

Scenario	Traditional .NET	Native AOT
Microservices	500ms cold start	10ms cold start → Cost reduction 🚀
Serverless/FaaS	1-2s AWS Lambda init	100-200ms init → Billing impact
Containers	800ms startup	10ms startup → K8s deployment efficiency
IoT/Embedded	~200MB footprint	~30MB footprint → Limited-resource devices
Desktop Apps	User waits 1+ second	Instant launch
Build-once, run-many	Size: ~100MB	Size: ~15MB → Deployment efficiency

Why Native AOT Matters (3)

The Trade-off

Traditional .NET

- ✓ Faster builds
- ✓ Runtime optimizations
- ✓ Full reflection support
- ✗ Slower startup
- ✗ Higher memory
- ✗ Larger deployments

Native AOT

- ✗ Slightly slower builds (AOT compilation)
- ✗ Limited reflection (needs trimming config)
- ✓ Instant startup
- ✓ Low memory
- ✓ Small deployments
- ✓ Predictable performance

Status: Production-Ready

Native AOT support:

-  Fully supported for console apps, Web APIs, workers
-  Trimming support improved for LINQ, Reflection
-  ASP.NET Core APIs have trimming annotations
-  .NET 10 adds more library compatibility

Limitations:

-  No dynamic code generation (Expression Trees have limits)
-  Reflection requires explicit configuration
-  Some NuGet packages may not be AOT-friendly (but improving)

Performance Comparison: .NET 10

Use Case	Traditional	Native AOT	Improvement
Console app startup	120ms	8ms	15x faster
Web API (systemd startup)	420ms	25ms	17x faster
Container cold boot	800ms	45ms	18x faster
Memory footprint	180MB	35MB	80% reduction
Binary size	110MB	22MB	80% reduction

When to Use Native AOT (1)

Perfect fit:

- Microservices and cloud functions (startup matters)
- Containerized workloads (density, cost)
- Serverless environments (billing impact)
- IoT and edge devices (resource constraints)
- Command-line tools (instant responsiveness)

When to Use Native AOT (2)

✗ Not ideal:

- Complex UI apps with plugins (dynamic loading)
- Apps with heavy reflection requirements
- Development environments (slower builds)
- Legacy codebases (extensive reflection)

Async Methods — The Problem

Optimize async methods by eliminating state machine overhead

```
// Traditional async — always generates state machine
public async Task<User> GetUserAsync(string id)
{
    var user = await _cache.GetAsync(id); // State machine generated
    return user ?? await _db.GetAsync(id); // Even if cache hits!
}
```

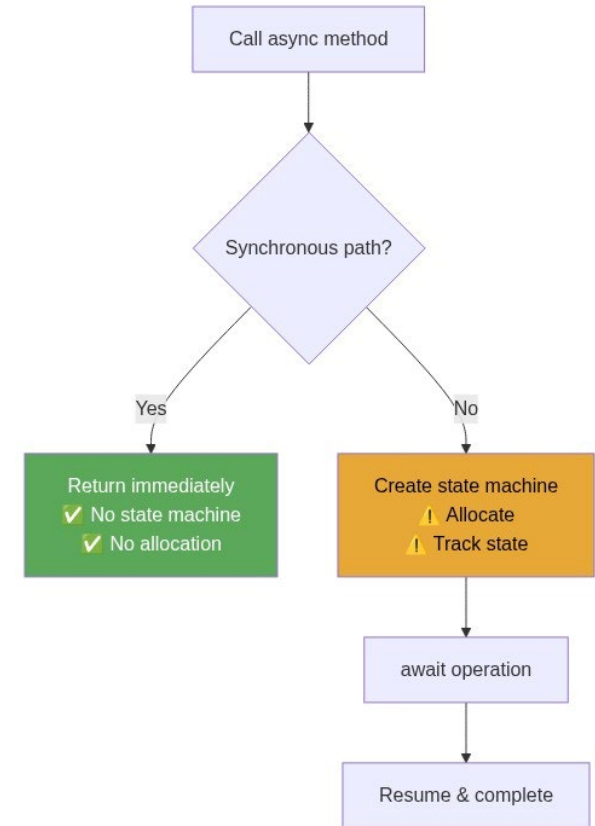
The problem:

-  State machine allocation (~200+ bytes)
-  State tracking overhead
-  Slower execution even when await completes synchronously

6. Runtime Async Optimization

C# 15 — Runtime async optimization:

- Async moves from libraries to the runtime
- Compiler automatically optimizes hot paths that complete synchronously!



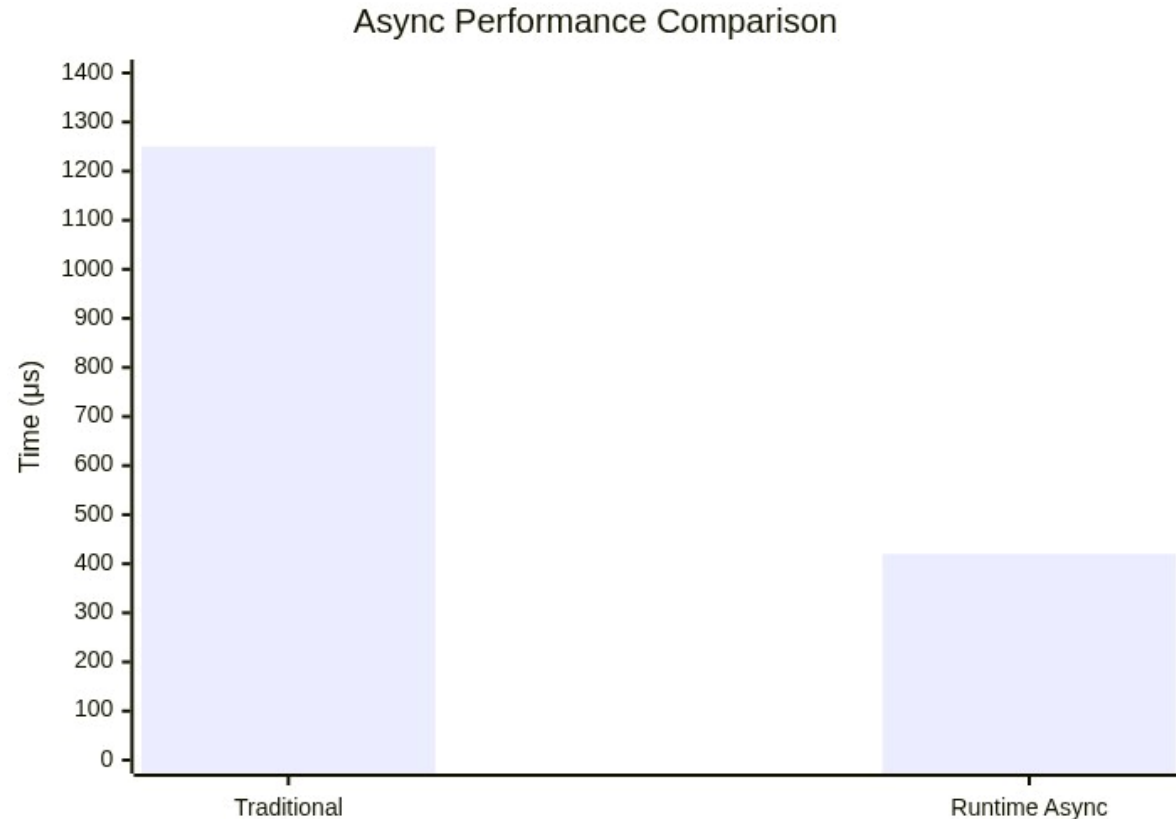
State machine generated only when actually needed

Runtime Async — Performance Impact

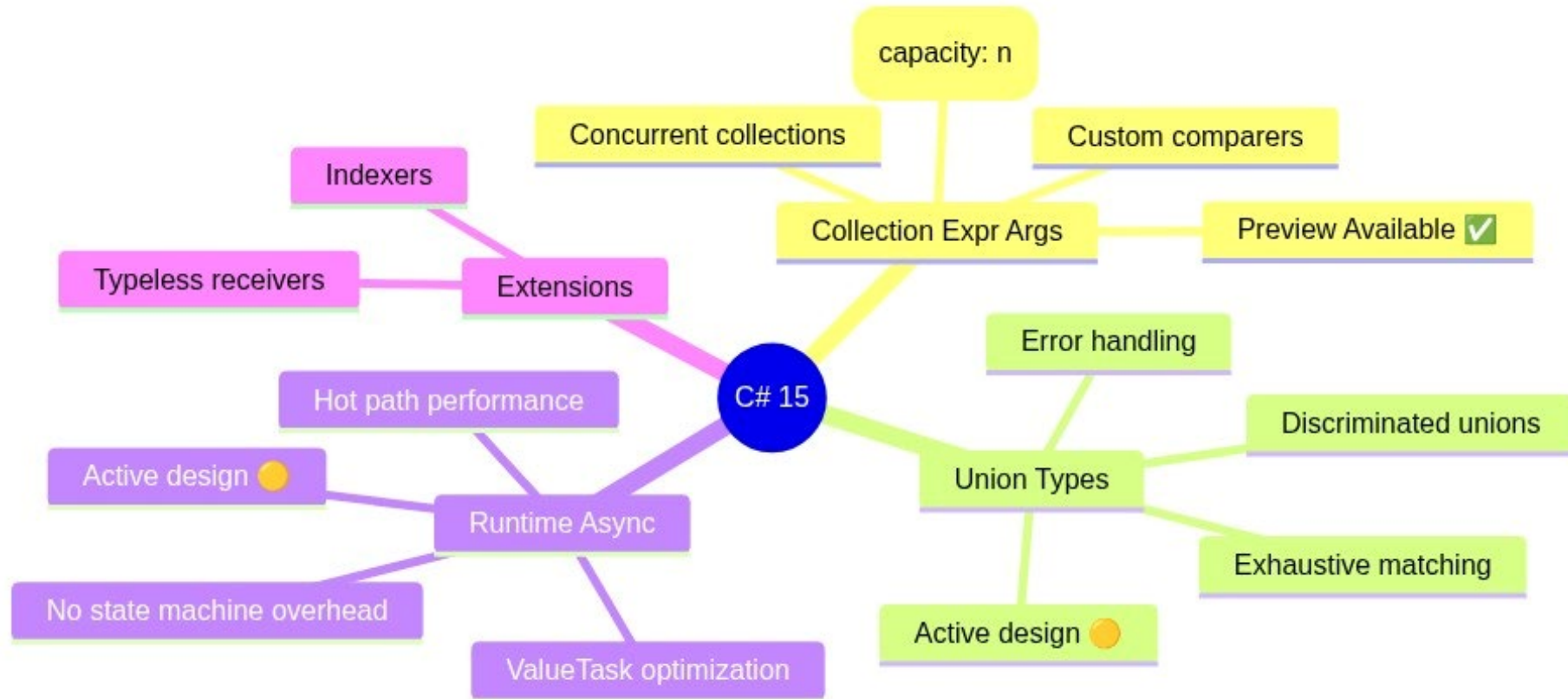
Benchmark: 10,000 calls with 90% cache hit rate

Method	Mean	Allocated
TraditionalAsync	1,250 μ s	2.1 MB
RuntimeAsync	420 μ s	210 KB

~66% faster, ~90% less memory for hot paths! 🚀



C# 15 at a Glance



Key takeaways

- **Post-Quantum Cryptography** — Protect your apps against future quantum threats
- **Extension Members** - Extend types with properties, operators, and static members for cleaner APIs
- **Span<T> & Implicit Conversions** — Zero-allocation performance without `.AsSpan()` calls
- **Native AOT** — Production-ready for microservices: 10-50x faster startup, 75% less memory
- **Runtime Async** — Auto-optimized hot paths eliminate state machine overhead
- **File-Based Apps** — Rapid prototyping with `dotnet run app.cs`

Q & A (1)

Common questions:

- *"Should we migrate to post-quantum algorithms now?"* — Start planning. Some industries (finance, government) should begin migration immediately.
- *"Are these algorithms production-ready?"* — Yes. NIST has standardized ML-KEM and ML-DSA.
- *"Can the new extension members access private fields" — No. They work like extension methods – public API only.*
- *"Do extension operators have performance overhead"* — No. Zero overhead, same as regular operators.

Q & A (2)

- *"Is runtime async a breaking change?"* — No. Opt-in with .NET 10, default with .NET 11. Existing code works unchanged.
- *"When does async runtime help most?"* — High-throughput APIs with caching layers.
- *"Are the language features breaking changes?"* — No — all features are backward compatible (exception next).
- *"field is a keyword — what if I have a variable named field?"* — Use @field to escape.

Thank You!

Enjoy Thrive Conference!

Happy coding with C# 14 and .NET 10!

Thrive Conference 2026

*Slides and samples: **github.com/CNILearn/thrive2026***

```
dotnet new console -f net10.0  
// Start exploring C# 14 and .NET 10 today!
```

Thank you

We greatly value your input. Please take a moment to complete our survey, accessible either through the Run.events app or by the link below:

<https://bit.ly/Thrive26Survey>

